RNA-seq NGS Pipeline

Author: Katarzyna Zielińska

Year**:** 2026

Introduction

RNA sequencing (RNA-seq) is a widely used method for studying gene expression and transcriptomic profiles. Before downstream analyses such as differential gene expression, RNA-seq data require several preprocessing and quality-control steps. This project presents an automated RNA-seq Next-Generation Sequencing (NGS) preprocessing pipeline implemented in Python. The pipeline processes paired-end RNA-seq data through quality control, adapter removal, quality trimming, read filtering, genome alignment, BAM processing, gene-level quantification, and quality reporting. The workflow was designed as a modular and reproducible pipeline that can be configured using a YAML configuration file and executed from the command line. The implementation uses Python to orchestrate established bioinformatics command-line tools.

Biological Question

The primary objective of this project is not to answer a specific biological question, but to establish a reproducible computational workflow for processing raw paired-end RNA-seq data before downstream transcriptomic analyses. The pipeline addresses the following methodological question: How can raw paired-end RNA-seq reads be systematically processed, quality-controlled, aligned to the human genome, and quantified at the gene level using an automated Python-based workflow?

Dataset

The pipeline is designed for paired-end RNA-seq data from *Homo sapiens*.
The example configuration uses:

- Organism: Homo sapiens
- Data type: Paired-end RNA-seq
- Example sample: SRR11365252

- Read 1: SRR11365252_1.fastq
- Read 2: SRR11365252_2.fastq
- Reference genome: Human genome
- Genome alignment: STAR
- Gene annotation: GENCODE v49
- Gene quantification: featureCounts

The pipeline supports multiple paired-end samples defined in the YAML configuration file. Raw sequencing data are not included in the repository because FASTQ files are typically too large for standard GitHub repositories.

Methods

The pipeline consists of seven main analytical stages.

1. Quality Control

Raw paired-end FASTQ files are evaluated using FastQC.
The quality-control step provides an initial assessment of:

- Per-base sequence quality
- GC content
- Sequence duplication
- Adapter contamination
- Overrepresented sequences

The generated reports are stored in the results/fastqc/ directory.

2. Adapter Clipping, Quality Trimming and Read Filtering

The fastp tool is used to preprocess the sequencing reads.
This step performs:

- Adapter removal
- Quality trimming
- Read filtering

The resulting trimmed FASTQ files are used as input for downstream genome alignment. HTML and JSON quality reports are also generated for each sample.

3. Genome Alignment

Trimmed reads are aligned to the reference genome using STAR.
The pipeline uses:

- Multi-threaded execution
- On-the-fly decompression of compressed FASTQ files
- Temporary STAR working directories
- Coordinate-sorted BAM output

The alignment results are stored in results/star/.

4. BAM Processing

Aligned reads are processed using SAMtools.
The pipeline performs:

- BAM sorting according to genomic coordinates
- BAM indexing

BAM indexing enables efficient access to genomic regions and prepares the alignment files for downstream analyses.

5. Gene Quantification

Gene-level read quantification is performed using featureCounts. Aligned reads are assigned to genomic features using a GTF annotation file. The resulting count table can subsequently be used for downstream differential gene expression analysis using tools such as DESeq2 or edgeR.

6. MultiQC Reporting

MultiQC aggregates quality-control results generated throughout the pipeline into a single interactive HTML report.
The report summarizes results from:

- FastQC
- fastp
- STAR

The final report is stored in results/multiqc/.

7. Pipeline Automation

The complete workflow is controlled by a Python script and executed in a predefined order:

1. FastQC
2. fastp
3. STAR
4. SAMtools Sort
5. SAMtools Index
6. featureCounts
7. MultiQC

The pipeline accepts a YAML configuration file through a command-line argument, allowing sample information, software paths, reference genome, annotation and computational resources to be specified independently from the pipeline code.

Software and Packages

The project uses:

- Python 3.11+
- PyYAML
- FastQC
- fastp
- STAR
- SAMtools
- featureCounts
- MultiQC
- Ubuntu / Linux
- WSL2

Python is used to orchestrate the external bioinformatics tools, while PyYAML is used to load the pipeline configuration. The script also uses Python modules including pathlib, argparse, subprocess, and logging.

Results

The pipeline generates a complete set of preprocessing and quality-control outputs for paired-end RNA-seq data.

The expected outputs include:

- FastQC quality reports
- Trimmed FASTQ files
- fastp HTML and JSON reports
- STAR alignment BAM files
- Coordinate-sorted BAM files
- BAM index files
- Gene-level count matrix
- MultiQC summary report

The pipeline therefore produces the main intermediate files required for subsequent RNA-seq analyses such as differential gene expression analysis. The workflow is designed to support multiple samples and provides a consistent directory structure for storing intermediate and final outputs.

Discussion

This project demonstrates how commonly used RNA-seq command-line tools can be integrated into a single reproducible Python workflow. Instead of executing FastQC, fastp, STAR, SAMtools, featureCounts and MultiQC independently, the pipeline coordinates these tools automatically and executes them in the correct analytical order. The use of a YAML configuration file separates pipeline parameters from the implementation itself. This makes it possible to modify sample information, reference genome, annotation and software paths without changing the main Python code. The pipeline also includes structured logging, allowing execution progress and commands to be recorded. The implementation uses Python's subprocess module to execute external bioinformatics programs and logging to report pipeline progress. An important feature of the workflow is its modular structure. Each major processing step is implemented as a separate Python function, making the pipeline easier to understand, test and extend. The current implementation focuses on preprocessing and quantification. It does not perform downstream differential expression analysis, pathway analysis or biological interpretation.

Conclusions

This project presents a reproducible Python-based RNA-seq preprocessing pipeline for paired-end sequencing data. The workflow integrates the major preprocessing steps required before downstream transcriptomic analysis:

- Quality control with FastQC
- Adapter removal and quality trimming with fastp
- Genome alignment with STAR
- BAM sorting and indexing with SAMtools
- Gene-level quantification with featureCounts
- Quality reporting with MultiQC

The project demonstrates practical skills in Python programming, Linux command-line tools, RNA-seq data processing, pipeline automation, configuration management and reproducible bioinformatics workflows. The resulting count matrix can serve as an input for downstream analyses such as differential gene expression using DESeq2 or edgeR.

Repository Structure

```
02_NGS_Pipeline_Python/
|
├── config.yaml
├── pipeline.py
├── requirements.txt
├── README.md
|
├── data/
|   ├── fastq/
|   ├── raw/
|   └── trimmed/
|
├── genome/
|   ├── annotation/
|   ├── reference/
|   └── star_index/
|
├── results/
|   ├── fastqc/
|   ├── fastp/
|   ├── bam/
|   ├── counts/
|   └── multiqc/
|
├── logs/
├── scripts/
└── tests/
```

References

1. Polański A., Podstawy bioinformatyki, Wydawnictwo Polsko-Japońskiej Wyższej Szkoły Technik Komputerowych, Warszawa, 2010.
2. Lesk A. M., Wprowadzenie do bioinformatyki, Wydawnictwo Naukowe PWN, Warszawa, 2019.
3. Wang X., Next-Generation Sequencing Data Analysis, 2nd ed., CRC Press, 2024.
4. Korpelainen E., Tuimala J., Somervuo P., Huss M., Wong G., RNA-seq Data Analysis: A Practical Approach, CRC Press, 2015.
5. Azad R. K. (ed.), Transcriptome Data Analysis, Methods in Molecular Biology, vol. 2812, Springer, 2024.
6. A Guide to Applied Machine Learning for Biologists, Springer, 2023.
7. Géron A., Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems, O'Reilly Media, 2025.
8. Alkhateeb A., Rueda L. (eds.), Machine Learning Methods for Multi-Omics Data Integration, Springer, 2023.
9. Ravarani C., Latysheva N., Deep Learning for Biology, O'Reilly Media, 2025.
10. Chollet F., Watson M., Deep Learning with Python, 3rd ed., Manning Publications, 2025.
11. Chollet F., Kalinowski T., Deep Learning with R, 3rd ed., Manning Publications, 2026.